

BLACKBOX

Workbook: a system flight recorder in TypeScript and Bun

How to use this workbook

Part	What it is	What to do with it
Step	One small task.	Do it. Nothing else until its check passes.
Hint	The method, concept or doc page that solves the step.	Read only if stuck for more than 15 minutes.
Check	A command or observation with the expected result.	Run it. If the result differs, the step is not done.
Stuck?	Common symptoms and where to look.	Match the symptom before asking for help.
Done when	The milestone's exit criteria.	Tick every box, then commit.

Rules: No step is skipped. No code is copied from anywhere; it is written from the hint and the docs. After every step: `bunx tsc --noEmit` and `bun run lint` are clean. Progress is tracked in the repository README. Cheat sheets at the back explain the theory behind each milestone.

Paths: Every command in this workbook runs from the repository root (BlackBox/) unless a step says otherwise. Where a command must run inside a package, it is wrapped in parentheses, for example `(cd packages/shared && bun add zod)`: the parentheses run it in a subshell, so the terminal stays in the root.

Reference: Repository and Commands

Layout	
BlackBox/	
package.json	"workspaces": ["packages/*"], no code
bun.lock	the only lockfile
tsconfig.json	
.oxlintrc.json	
packages/	
agent/	@blackbox/agent (depends on @blackbox/shared)
ingest/	@blackbox/ingest (depends on @blackbox/shared)
shared/	@blackbox/shared
dashboard/	added in Milestone 13

Task	Command
Run the agent	bun run packages/agent/index.ts
Type check everything (Bun does not)	bunx tsc --noEmit
Lint	bun run lint
Run tests	bun test
Add a dependency to one package	(cd packages/<name> && bun add <pkg>)
Link a workspace package	add "@blackbox/shared": "workspace:*" to dependencies, then bun install from root
Commit	git add . && git commit -m "..."

Architecture

agent (every machine)		ingest server (one host)		dashboard
+-----+	WebSocket	+-----+	HTTP	+-----+
collectors	----->	auth, validate	<-----	Next.js
ring buffer (disk)	samples,	store (SQLite)	+ WS	history,
reconnect, replay	events	rules, fan-out	----->	live
+-----+		+-----+		+-----+

Core rule: The agent reads, timestamps, buffers and sends. It never interprets. Thresholds, incidents and comparisons belong to the server.

MILESTONE 1 · CHECKPOINT

Memory Reader

Goal: Confirm the finished state before building on it.

1.1**Verify it runs**

Run the agent for 30 seconds.

Check One `MemorySample` per interval with five fields plus `sampledAtMs`. `Ctrl+C` stops it.

1.2**Verify error handling**

Add "MemBanana" to the required fields and run again.

Check One line per interval: Failed to read memory: meminfo is missing: MemBanana. The agent keeps running. Remove it afterwards.

1.3**Name the interval**

Replace the bare number in `setInterval` with a constant `SAMPLE_INTERVAL_MS` at the top of the file.

Hint Constants that configure behaviour are written in `UPPER_SNAKE_CASE` by convention.

Check Changing the constant changes the output rate.

1.4**Clean checks**

Run the type checker and the linter.

Check `bunx tsc --noEmit` and `bun run lint` print no errors.

Answer without looking

- ☐ Why does `parseInt("MemTotal: 7915828 kB")` return `NaN`?
- ☐ Why does the empty last line of the file not produce an entry?
- ☐ What would `for...in` produce instead of `for...of`?
- ☐ Why is the file read inside the `try`?
- ☐ Why does `parseMemoryInfo` throw instead of catching its own error?

MILESTONE 2

Parser Tests

Goal: Prove the parser works with fixed inputs, without the live file.

You will learn: modules and export · test fixtures · bun test · testing that code throws

Before you start: Read the Bun test runner page: <https://bun.sh/docs/cli/test>

2.1 Create the folders

Inside packages/agent, create src/collectors/ and fixtures/.

Check `ls packages/agent` shows both.

2.2 Move the parser into a module

Move `parseMemoryInfo` into `src/collectors/memory.ts` and export it. `index.ts` imports it and keeps reading, scheduling and logging.

Hint A module must not start timers when imported. Only `index.ts` may call `setInterval`.

Check The agent runs exactly as before.

2.3 Save a fixture

Store a real snapshot of the file: `cp /proc/meminfo packages/agent/fixtures/meminfo.txt`

Check `head -3 packages/agent/fixtures/meminfo.txt` shows `MemTotal`, `MemFree`, `MemAvailable`.

2.4 First test

Create `memory.test.ts` next to `memory.ts`. Read the fixture, parse it, expect `totalKb`, `freeKb` and `availableKb` to equal the numbers in the fixture (`head -3` shows them).

Hint `test(name, async () => {...})` from `bun:test`. Build the fixture path from `import.meta.dir`, the test file's own folder (already an absolute path), plus `../../../../fixtures/meminfo.txt`. Matcher: `toBe`.

Check `bun test` reports 1 pass.

2.5 Missing-field test

Create a fixture without `MemAvailable`: `grep -v '^MemAvailable' packages/agent/fixtures/meminfo.txt > packages/agent/fixtures/meminfo-missing.txt`. Write a second test: parsing it throws an error that mentions `MemAvailable`.

Hint `toThrow` needs a function it can call itself: pass an arrow function that calls the parser, not the parser's result. Never redirect output into the file being read; the shell empties it first.

Check `bun test` reports 2 pass.

2.6 No trailing newline should parse

Create the fixture: `head -c -1 packages/agent/fixtures/meminfo.txt > packages/agent/fixtures/meminfo-no-newline.txt`. Write a test that parses it and checks `totalKb`.

Hint Same structure as 2.4. Both fixtures come from the same snapshot, so the number is identical.

Check `bun test` reports 3 pass.

2.7

Empty input should throw

Write a test that passes "" directly to the parser and expects a throw mentioning `MemTotal`.

Hint Same structure as 2.5, without a fixture and without `async`: nothing is read from disk.

Check `bun test` reports 4 pass.

2.8

Prove the tests can fail

Comment out the throw line in the parser (Neovim: `gcc`) and run the tests. Then restore it.

Hint Look at the received value in the failure: fields typed `number` are `undefined`. That is what the `!` assertion hides when its guarding check is missing.

Check Without the throw: 2 pass, 2 fail (missing-field and empty input). Restored: 4 pass.

Stuck?

If you see	Look at
Test cannot import the parser	Missing <code>export</code> , or wrong relative path in the import.
Timer starts when tests run	<code>setInterval</code> is inside the module instead of <code>index.ts</code> .
<code>toThrow</code> test fails although the parser throws	The parser was called directly instead of inside a function passed to <code>expect</code> .
Fixture not found	Paths are relative to where <code>bun test</code> runs. Build the path from <code>import.meta.dir</code> .
Path contains the home folder twice	<code>import.meta.dir</code> is already absolute. Append only the relative part.
<code>toThrow</code> test fails with a different message	Typo in the expected substring (compare letter by letter).

Done when

- ☐ `bun test` passes.
- ☐ Importing `memory.ts` starts nothing.
- ☐ Removing the throw makes a test fail.
- ☐ Committed with a message that names the milestone.

MILESTONE 3

CPU Usage

Goal: Report CPU utilisation in percent for each sample interval.

You will learn: counters versus gauges · state between samples · pure calculation functions

Before you start: Read Cheat sheet A (gauges and counters). Docs:

https://man7.org/linux/man-pages/man5/proc_stat.5.html

3.1 Observe the data

Run `head -1 /proc/stat`, wait one second, run it again. Compare the numbers.

Check Every number is equal or larger in the second reading. Nothing ever decreases.

3.2 Define the type

In `@blackbox/shared`, define a type for the eight counters: `user`, `nice`, `system`, `idle`, `iowait`, `irq`, `softirq`, `steal`.

Check The type is importable from the agent.

3.3 Write the parser

In `src/collectors/cpu.ts`, write `parseCpuStat(text)`. Take the first line, split on whitespace, drop the `cpu` label, convert the eight values, check they exist.

Hint Same technique as `meminfo`: `split`, `parseInt`, `required-field` check. The line has no colons.

Check Logging the result once shows eight numbers matching `head -1 /proc/stat`.

3.4 Test the parser

Save a fixture: `cp /proc/stat packages/agent/fixtures/stat.txt`. Write a test like in Milestone 2.

Check `bun test` passes.

3.5 Write the calculator

Write a pure function `cpuUsagePercent(previous, current)` returning a number.

Hint Formula in Cheat sheet A. `idle = idle + iowait`. `busy = user + nice + system + irq + softirq + steal`.

Check Test: previous idle 100, total 200; current idle 150, total 300. Result: 50.

3.6 Keep the previous reading

In the agent, keep the last counters in a variable outside the recording function. The first round stores counters and reports nothing.

Check First interval: no CPU value. From the second: a percentage every interval.

3.7 Verify under load

Start `yes > /dev/null &`, watch the value, stop it with `kill %1`. Compare with `top`.

Check Usage rises by about 100 divided by the number of cores, then falls back.

Stuck?

If you see	Look at
Value never changes	Computed from one reading (average since boot) instead of the difference of two.
NaN or Infinity	Total difference is zero. Guard against dividing by zero.
Value above 100	guest and guest_nice were added; they are already inside user and nice.

Done when

- ☐ Parser and calculator tested.
- ☐ Agent prints CPU % from the second interval on.
- ☐ Load test behaves as expected.
- ☐ Committed with a message that names the milestone.

MILESTONE 4

More Collectors

Goal: Add load average, network throughput and temperature as independent collectors.

You will learn: optional data · reading directories · failure isolation between collectors

Before you start: Read Cheat sheet A. Docs: <https://man7.org/linux/man-pages/man5/proc.5.html> (sections loadavg, net/dev)

4.1 Load average

Parse `/proc/loadavg`: the first three numbers are the 1, 5 and 15 minute averages.

Hint `parseFloat`, not `parseInt`: these values have decimals.

Check Values match `uptime`. Parser has a test.

4.2 Network parser

Parse `/proc/net/dev`. Skip the two header lines. Per interface: name, received bytes (first value), transmitted bytes (ninth value). Exclude `lo`.

Hint The colon after the interface name can touch the first number with no space. The colon-replacement trick from Milestone 1 handles it.

Check Test with a fixture passes; output lists your real interfaces.

4.3 Network rates

Turn the byte counters into bytes per second using the pattern from Step 3.6.

Hint A negative difference means the counter was reset. Report nothing for that round.

Check Downloading a large file raises the received rate visibly.

4.4 Temperature

List `/sys/class/thermal/`, read each zone's type and temp, divide by 1000.

Hint `readdir` from `node:fs/promises`. A machine or VM without thermal zones returns an empty list, not an error.

Check Values are plausible (30 to 90 °C) and match sensors if installed.

4.5

Isolate failures

Run all collectors in the same interval so that one failing collector does not stop the others.

Hint Each collector gets its own `try...catch`, or look at `Promise.allSettled` on MDN.

Check Point the temperature path at a folder that does not exist: temperature logs an error, everything else continues.

Stuck?

If you see	Look at
Network numbers shifted by one column	The interface name and first number were not separated.
Temperature 45000	Value is in millidegrees. Divide by 1000.
Whole round lost when one file is missing	Collectors share one <code>try</code> . Give each its own.

Done when

- ☐ Four collectors, each with a test.
- ☐ A broken collector does not affect the others.
- ☐ Committed with a message that names the milestone.

MILESTONE 5

Local Recording

Goal: Store samples on disk in a fixed-size buffer that survives crashes.

You will learn: ring buffers · batching and flushing · `fsync` · process crash versus power loss

Before you start: Read Cheat sheet B (durability). Decide the format with the table below and write the decision in the agent's README.

Format	Strength	Weakness
JSON Lines segment files (one file per N minutes)	Simple. Inspect with <code>tail</code> . Delete oldest file to free space.	Replay needs a position (file + line).
Local SQLite (<code>bun:sqlite</code>)	Transactions, indexes, replay by sequence number.	More to learn: WAL mode, schema.

5.1

Batch in memory

Collect samples in an array. Every `FLUSH_INTERVAL_MS`, write the batch to disk and clear it.

Hint Choose the flush interval as the maximum data you accept losing on power failure. Write the number down.

Check Logging each flush shows several samples per write, not one.

5.2

Write to disk

Implement the chosen format. Segment files: name them by start time. SQLite: one table with sequence, kind, sampledAtMs, payload.

Hint Docs: <https://bun.sh/docs/api/sqlite>

Check Stop the agent, inspect the data with `tail` or `sqlite3`: samples are there.

5.3**Enforce capacity**

Delete data older than a limit (for example 24 hours) during each flush.

Check Set the limit to 2 minutes temporarily: old data disappears.

5.4**Crash test**

Run the agent, find its process with `pgrep -f index.ts`, kill it with `kill -9 <pid>`.

Check Data up to the last flush is on disk. At most one flush interval is missing.

5.5**Reader**

Write a function that returns all samples after a given position. Milestone 10 depends on it.

Check Test: after position N, it returns exactly the samples written after N.

Stuck?

If you see	Look at
Thousands of tiny files	Writing per sample instead of per batch.
Everything lost on crash	Batch held in memory too long; flush interval too high.
Disk usage keeps growing	Retention not running, or comparing times in different units.

Done when

- ☐ `kill -9` loses at most one flush interval.
- ☐ Disk usage stays flat over a day.
- ☐ Reader tested.
- ☐ Committed with a message that names the milestone.

MILESTONE 6

Lifecycle Events

Goal: Record how the previous session ended: clean stop, agent crash, or unclean reboot.

You will learn: signals · marker files · boot ID · events as a data type

Before you start: Docs: <https://nodejs.org/api/process.html#signal-events>

6.1 Read the boot ID

Read `/proc/sys/kernel/random/boot_id` at startup.

Check `cat` the file twice: same value. After a reboot: a different value.

6.2 Handle signals

On `SIGTERM` and `SIGINT`: flush the buffer, write a marker file containing the boot ID, exit.

Hint `process.on(...)`. Exit only after the flush has finished.

Check `Ctrl+C` creates the marker file.

6.3 Classify at startup

Read the marker and the current boot ID, decide which row of the table applies, record an event, delete the marker.

Hint Keep the decision in a pure function (marker present, boot ID changed) that returns the event kind, and test it.

Check A test covers all four combinations of the table.

6.4 Verify on a real machine

Produce each case with the method in the last column and read the recorded event.

Check All four rows produce the expected event.

Marker	Boot ID changed	Event	How to produce it
yes	no	clean restart	<code>Ctrl+C</code> , start again
yes	yes	clean reboot	<code>Ctrl+C</code> or <code>systemd stop</code> , then reboot
no	no	agent crashed	<code>kill -9</code> , start again
no	yes	unclean reboot	Hard reset of a test VM in Proxmox

Stuck?

If you see	Look at
Marker never written	Process exits before the async flush completes.
Every start looks like a crash	Marker written to a relative path that differs between runs.

Done when

- ☐ All four rows produced and recorded correctly.
- ☐ Committed with a message that names the milestone.

MILESTONE 7

Shared Protocol

Goal: Define every message once, as runtime schemas with types derived from them.

You will learn: runtime validation · `z.infer` · discriminated unions · single source of truth

Before you start: Read Cheat sheet C (types versus validation). Docs: <https://zod.dev>

7.1 Install zod

(`cd packages/shared && bun add zod`)

Check zod appears in `packages/shared/package.json`.

7.2 Schemas for samples

Write schemas for memory, CPU, load, network and temperature samples. Export the types with `z.infer`. Delete the hand-written interfaces.

Check The agent compiles unchanged: `bunx tsc --noEmit` is clean.

7.3 Message schemas

Define the messages below. Combine them into one union keyed by `type`.

Hint zod has a dedicated function for unions with a shared discriminant field.

Check Hovering the union type shows all five message shapes.

type	Direction	Fields
hello	agent to server	agentId, token, protocolVersion, bootId
welcome	server to agent	lastSeq
sample	agent to server	seq, kind, sampledAtMs, data
event	agent to server	seq, kind, sampledAtMs, detail
ack	server to agent	seq

7.4 Valid messages pass

Write a test that parses one valid example of each message type.

Hint `safeParse` returns a result object with a `success` field instead of throwing.

Check `bun test` passes; every message type has a valid example.

7.5 Invalid messages are rejected

Write one test per failure: an unknown type, a missing field, and a string where a number belongs.

Hint Each test changes exactly one thing in a valid example, so a failure points to one cause.

Check `bun test` passes; three rejection tests.

Done when

- ☐ No hand-written type duplicates a schema.
- ☐ Invalid messages are rejected in tests.
- ☐ Committed with a message that names the milestone.

MILESTONE 8

Ingest Server

Goal: Accept agent connections over WebSocket and authenticate them.

You will learn: Bun .serve WebSockets · connection state machines · timing-safe comparison · close codes

Before you start: Docs: <https://bun.sh/docs/api/websockets>. Test client: `bunx wscat -c ws://localhost:7070/agent`

8.1 Accept connections

In packages/ingest, start a server that upgrades requests on /agent to WebSocket. Log open and close.

Check Connecting with wscat logs open; Ctrl+C in wscat logs close.

8.2 Connection state

Give each connection a state: Authenticating, Streaming, Closed. In Authenticating, only hello is accepted.

Hint Bun lets you attach a data object to each connection during the upgrade.

Check Sending a sample before hello closes the connection.

8.3 Validate messages

Validate every incoming message with the shared schemas before acting on it.

Hint JSON.parse can throw and safeParse can fail. Both must be handled without crashing the server.

Check Sending hello world or {} with wscat is logged as invalid; the server keeps running.

8.4 Authenticate

Check the token in hello against a config file mapping agentId to token.

Hint Compare tokens with timingSafeEqual from node:crypto. Close with code 1008 on failure.

Check Wrong token: closed with 1008. Correct token: log shows the agent ID.

8.5 Hello timeout

Close any connection that has not sent a valid hello within 5 seconds.

Check Connect with wscat and send nothing: closed after 5 seconds.

8.6 Log close codes

Record the close code of every connection.

Check Clean close shows 1000 or 1001. Killing the client process shows 1006.

Stuck?

If you see	Look at
Server crashes on bad JSON	JSON.parse not inside try, or schema parse used instead of safeParse.
Upgrade never happens	Path mismatch, or the fetch handler returns a response instead of upgrading.

Done when

- ☐ Unauthenticated connections are always closed.
- ☐ Server survives any input sent with wscat.
- ☐ Committed with a message that names the milestone.

MILESTONE 9

Agent Connection

Goal: Stream from agent to server and survive server outages.

You will learn: WebSocket client · exponential backoff with jitter · dead-connection detection

Before you start: Docs: <https://developer.mozilla.org/en-US/docs/Web/API/WebSocket>

9.1

Connect and hello

The agent opens a WebSocket to the server and sends hello on open.

Check Server logs the agent as authenticated.

9.2

Send samples

After welcome, send each new sample as a sample message.

Check Server logs incoming samples at the sample rate.

9.3

Reconnect

On close, reconnect after a delay: base times two to the power of the attempt, capped at 30 seconds, plus a random amount. Reset the attempt count after a successful hello.

Hint Without the random part, all agents reconnect at the same instant when the server returns.

Check Stop the server: the agent logs growing, uneven delays. Start it: the agent reconnects.

9.4

Recording is independent

Samples always go to the local buffer. Sending reads from the buffer. A dead connection never stops recording.

Check With the server down for 2 minutes, the local buffer keeps growing.

9.5

Detect dead connections

Make sure a connection whose peer vanished is noticed.

Hint WebSocket ping/pong. Check Bun's server options for built-in pings and idle timeouts.

Check Pull the network (or firewall the port): the connection is declared dead within the timeout.

Stuck?

If you see	Look at
Reconnect storm, many connections at once	Old socket not closed before opening a new one.
Delay stays at the maximum forever	Attempt count never reset after success.

Done when

- ☐ Server restarts do not interrupt local recording.
- ☐ Agent reconnects on its own.
- ☐ Committed with a message that names the milestone.

MILESTONE 10

Resume Without Gaps

Goal: After any outage, the server holds every sample exactly once.

You will learn: sequence numbers · acknowledgements · idempotent writes · backpressure

Before you start: Read Cheat sheet D (delivery).

10.1

Sequence numbers

Give every sample and event a sequence number that increases by one and survives agent restarts.

Hint Store the last sequence number in the local buffer, not in memory.

Check Restart the agent: numbering continues instead of starting at 1.

10.2

Welcome with lastSeq

The server answers hello with the highest sequence it has stored for this agent.

Check Log shows the value the server sends.

10.3

Replay

After welcome, the agent sends everything after lastSeq from the buffer, then continues live.

Hint This is the reader from Step 5.5.

Check Stop the server for 3 minutes, restart: the backlog arrives, then live samples.

10.4

Idempotent storage

The server ignores a sample whose agent and sequence it already has.

Check Replaying the same range twice does not create duplicates.

10.5

Backpressure

If the socket cannot keep up, pause sending instead of queueing without limit.

Hint Check what send returns in Bun's WebSocket docs.

Check Memory use of the agent stays flat during a large replay.

Done when

- ☐ After an outage, sequence numbers on the server have no gaps and no duplicates.
- ☐ Committed with a message that names the milestone.

MILESTONE 11

Server Storage

Goal: Persist all data on the server with bounded disk use.

You will learn: schema design · WAL mode · Drizzle · retention

Before you start: Docs: <https://orm.drizzle.team/docs/get-started-sqlite> and <https://www.sqlite.org/wal.html>

11.1 Schema by hand

Write the tables in raw SQL first: agents, samples, events. Samples: agent, kind, seq, sampledAtMs, receivedAtMs, payload. Unique on agent + seq.

Check `sqlite3` shows the tables and the unique index.

11.2 WAL mode

Enable write-ahead logging so the dashboard can read while the server writes.

Check `PRAGMA journal_mode;` returns `wal`.

11.3 Move to Drizzle

Recreate the schema with Drizzle and replace the raw queries.

Check Type errors appear in queries when a column is renamed.

11.4 Index and query

Add an index on agent + sampledAtMs. Query one agent over one hour.

Check The query returns in milliseconds.

11.5 Retention

Delete samples older than a configurable age on a schedule.

Check With a short test age, old rows disappear.

Done when

- ☐ Server restart loses nothing.
- ☐ Disk use bounded by the retention setting.
- ☐ Committed with a message that names the milestone.

MILESTONE 12

Rules

Goal: Turn stored data into events: spikes, silence, skew, crashes.

You will learn: thresholds with duration · hysteresis · silence detection

Before you start: Install a load generator: `sudo pacman -S stress-ng`

12.1 Threshold with duration

Raise an event when memory use stays above a configurable threshold (default 95%) for 30 seconds.

Hint Test with a lowered threshold. Pushing a real machine to 95% risks the OOM killer ending other programs.

Check Threshold set to 80%: `stress-ng --vm 1 --vm-bytes 60% --timeout 60s` produces one event.

12.2 Hysteresis

End the event only when usage falls below a lower limit (for example 5 points under the threshold).

Hint Without it, a value oscillating around 95% creates dozens of events.

Check One event with a start and an end, not many short ones.

12.3 Silence

Raise an event when an agent sends nothing for several intervals.

Check Stop one agent: one silence event.

12.4 Clock skew

On live data, compare `receivedAtMs` with `sampledAtMs`. Raise an event above a limit.

Hint Add a temporary clock offset setting to the agent for testing.

Check An offset of 5 minutes produces a skew event.

12.5 Lifecycle events

Store the agent's crash and reboot events from Milestone 6 as server events.

Check `kill -9` on an agent shows up as an event on the server after restart.

Done when

- ☐ Each rule tested with a deliberate trigger.
- ☐ Committed with a message that names the milestone.

MILESTONE 13

Dashboard

Goal: Show agents, history, events and live data in the browser.

You will learn: HTTP API with Hono · one owner of the database · chart density

Before you start: Docs: <https://hono.dev>, <https://nextjs.org/docs>, <https://ui.shadcn.com/charts>

13.1 History API

Add HTTP routes to the ingest server: list agents, samples for one agent in a time range, events.

Hint The dashboard never opens the database itself. The ingest server owns it.

Check curl on each route returns JSON.

13.2 Create the app

(cd packages && bunx create-next-app@latest dashboard). The name must be lowercase. Add @blackbox/shared as a workspace dependency.

Check The dev server starts and imports a type from shared.

13.3 Agent list

Page listing agents with last-seen time.

Check Stopping an agent changes its last-seen time within one interval.

13.4 One graph

Memory over the last hour for one agent.

Hint Convert epoch milliseconds to local time only in the view.

Check The graph matches the values in the database.

13.5 Event timeline

List events; selecting one shows all metrics for that machine in the 5 minutes before it.

Check The stress-ng event from Milestone 12 shows the memory rise before it.

13.6 Live updates

The browser opens a WebSocket to the ingest server and appends new samples.

Check The graph moves without reloading.

Stuck?

If you see	Look at
Browser freezes on large ranges	Too many points. Downsample on the server.
Times off by one or two hours	Converted to local time twice, or stored in local time.

Done when

- ☐ Any event can be inspected with its preceding 5 minutes.
- ☐ Committed with a message that names the milestone.

MILESTONE 14

Deployment

Goal: Every machine reports after boot without anyone logging in.

You will learn: single-file executables · systemd services · least privilege · time sync

Before you start: Docs: <https://bun.sh/docs/bundler/executables> and <https://www.freedesktop.org/software/systemd/man/latest/systemd.service.html>

14.1 Configuration file

Move interval, server URL, agent ID and token path out of the code into a config file.

Check Changing the config changes behaviour without rebuilding.

14.2 Build the binary

```
bun build packages/agent/index.ts --compile --outfile dist/blackbox-agent
```

Check The binary runs on a machine without Bun installed.

14.3 Service user

```
sudo useradd --system --no-create-home blackbox. Token file readable only by this user.
```

Check `ls -l` on the token file shows mode 600 and owner blackbox.

14.4 systemd unit

Write a unit that runs the binary as the service user and restarts it on failure.

Hint Keys to look up: ExecStart, User, Restart, RestartSec, WantedBy.

Check `systemctl status` shows active; `kill -9` on the process: systemd restarts it.

14.5 Time sync

Enable chrony or systemd-timesyncd on every machine.

Check `timedatectl status` shows the clock as synchronized.

14.6 Secure transport

Route agent traffic over WireGuard or Tailscale, or put TLS in front of the server.

Check Traffic between machines is not readable with `tcpdump`.

14.7 Roll out

One second machine first. Only when two machines work, all the rest.

Hint Inside LXC containers several `/proc` files show the host's values unless `lxcfs` is active. Compare against the Proxmox host.

Check Cold reboot of every machine: all report within one minute.

Done when

- ☐ All machines report after a cold reboot.
- ☐ Agents run unprivileged.
- ☐ Committed with a message that names the milestone.

Cheat Sheet A: Reading the System

Concept	Short version
/proc	Virtual files. The kernel generates the content at the moment of reading. Two reads, two snapshots.
Gauge	A current level (memory in use, temperature, load). One reading is meaningful.
Counter	Only increases since boot (CPU time, bytes sent). Meaningful only as the difference of two readings.
MemAvailable	What a new program can get without swapping. Primary memory metric. $\text{Used} = \text{Total} - \text{Available}$.
MemFree	Untouched RAM. Low on healthy systems because the kernel fills free RAM with cache.
Shmem	Shared memory and tmpfs. Counted in Cached, but cannot be dropped.
Dirty	Written by programs, not yet on disk. Lost on power failure.
CPU usage	$\text{idle} = \text{idle} + \text{iowait}$; $\text{busy} = \text{user} + \text{nice} + \text{system} + \text{irq} + \text{softirq} + \text{steal}$; $\text{total} = \text{idle} + \text{busy}$. $\text{usage \%} = (\Delta\text{total} - \Delta\text{idle}) / \Delta\text{total} \times 100$.

Source	Kind	Unit
/proc/meminfo	gauge	kB
/proc/stat (first line)	counter	ticks
/proc/loadavg	gauge	none
/proc/net/dev	counter	bytes, packets
/sys/class/thermal/thermal_zone*/temp	gauge	millidegree C
/proc/sys/kernel/random/boot_id	identifier	UUID

Cheat Sheet B: Data and Durability

Rule	Why
Store raw values, derive percentages	A percentage loses information and freezes today's formula into old data.
Agent stamps sampledAtMs	Backlog sent after an outage must keep its real time.
Server stamps receivedAtMs	The difference reveals backlog and wrong clocks.
Time as epoch milliseconds	Date does not survive JSON. Numbers sort, subtract, store as INTEGER.
Units in names	totalKb, sampledAtMs: no guessing seconds vs milliseconds.
Identity per connection	Sent once in hello, not in every sample.
A finished write call is not on disk	Data sits in the page cache until flushed. fsync forces it.
Process crash vs power loss	Process crash: the kernel still flushes written data. Power loss: unflushed data is gone.

Cheat Sheet C: TypeScript for a C# Developer

C#	TypeScript	Watch out
string?	string undefined	undefined = never set, null = deliberately empty.
s.Split(sep, 2)	s.split(sep, 2)	JS discards everything after the limit.
int.Parse	Number.parseInt	No exception: returns NaN if no leading digit.
foreach (x in list)	for (const x of list)	for...in gives indices as strings.
Dictionary<string,int>	Record<string, number>	Lookups return number undefined.
LINQ Where	filter	Takes an arrow function returning a boolean.
`\${x}`	backticks with \${x}	Backticks, not quotes.
catch (Exception e)	catch (error)	error is unknown; narrow with instanceof Error.
x!	x!	Unchecked promise to the compiler. Comment why it is safe.
async Task	Promise<void>	Unhandled rejections end the Bun process.
_ = DoAsync();	void doAsync()	Only safe if the function catches its own errors.
DTO class	interface / type	Erased at runtime. Network input needs validation (zod).

Rule	Meaning
Bun does not type check	Red squiggles can coexist with a running program. Run <code>bunx tsc --noEmit</code> .
Narrowing	After <code>if (x !== undefined)</code> or an early <code>continue/return/throw</code> , TypeScript treats <code>x</code> as defined.
Return type position	Between the parameter list and the opening brace.
return ends the function	Several values are returned as one object.
Parser signals, caller decides	Parsers throw. The calling loop catches, logs, and continues.

Cheat Sheet D: Delivery

Concept	Short version
Store-and-forward	Record locally always; send when possible.
Backoff with jitter	Delay = $\min(\text{cap}, \text{base} \times 2^{\text{attempt}}) + \text{random}$. Prevents all agents reconnecting at once.
Half-open connection	TCP can look alive after the peer lost power. Ping/pong detects it.
Close codes	1000/1001 deliberate, 1006 died without a close frame, 1008 policy violation.
Sequence number	Per agent, +1 per message, survives restarts.
Ack / lastSeq	The server says what it has; the agent replays the rest.
Idempotent write	Unique agent + seq: storing twice changes nothing.
Backpressure	When the receiver is slow, the sender slows down instead of queueing forever.

Debugging Table

Symptom	Likely cause	Fix or inspect
NaN from parseInt	Text does not start with a digit.	Log the exact input string.
Keys are "0", "1", "2"	for...in on an array.	for...of.
Only the first field returned	return inside the loop.	Move return after the loop.
Return type ends in undefined	A path reaches no return.	Every path returns or throws.
"Declaration or statement expected"	Type annotation outside the signature.	Position of : Type.
"}' expected"	Unbalanced braces.	Neovim: cursor on {, press %.
"'catch' or 'finally' expected"	try without catch.	Pair every try.
"Unreachable code detected"	Code after return or throw.	Return one object.
Editor error, program runs	Bun ignores types.	bunx tsc --noEmit.
Process exits on one bad read	Unhandled rejection.	try...catch inside the async callback.
bun: command not found	PATH set in the wrong shell file.	~/ .bashrc, new terminal.
Workspace import fails	Dependency not declared or file not saved.	cat package.json, bun install.
Linters ignores rules	Config file name misspelled.	.oxlintrc.json.
Stack trace panel in editor	Language server crashed.	:LspRestart.

References

Topic	URL
Bun docs	https://bun.sh/docs
Bun file I/O	https://bun.sh/docs/api/file-io
Bun test	https://bun.sh/docs/cli/test
Bun WebSockets	https://bun.sh/docs/api/websockets
Bun SQLite	https://bun.sh/docs/api/sqlite
TypeScript Handbook	https://www.typescriptlang.org/docs/handbook/intro.html
MDN JavaScript	https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference
proc(5)	https://man7.org/linux/man-pages/man5/proc.5.html
zod	https://zod.dev
Drizzle	https://orm.drizzle.team
Hono	https://hono.dev
Oxlint	https://oxc.rs/docs/guide/usage/linter.html